

Software Requirements Specification (SRS)

Project: FreelanceTrace – The AI-Powered Trust & Traceability Interface

Version: 1.2.0

Date: March 29, 2026

Status: Performance Baseline for Project Review

1. Introduction

1.1 Purpose

Definition of the technical and functional requirements for **FreelanceTrace**, the next-generation freelance management system. This document is designed to be parsed by the automated **Gemini SRS Extractor** and tracked via **SBERT Semantic Traceability**.

1.2 Product Vision

FreelanceTrace eliminates the "Trust Gap" between clients and freelancers. It uses code-level traceability to ensure that "Done" actually means "Done" according to the original contract.

2. Functional Requirements

2.1 Identity & Access Management (IAM)

- **REQ-01 (High):** System **shall** support **OAuth 2.0** and **JWT-based** authentication for secure, stateless user logins.
- **REQ-02 (Medium):** The backend **must** implement a **Role-Based Access Control (RBAC)** system separating 'Clients' from 'Freelancers'.
- **REQ-03 (Medium):** Two-factor authentication (2FA) **shall** be available as an opt-in security layer for all financial transactions.
- **REQ-04 (High):** Profile management **shall** allow users to link their GitHub accounts directly for repository-level verification.

2.2 Project & Bid Lifecycle

- **REQ-05 (High):** Clients **shall** be able to post projects with a defined budget range, duration, and specific technical stack tags.

- **REQ-06 (High):** Freelancers **must** be able to submit detailed proposals (bids) specifying an exact bid amount and estimated timeline.
- **REQ-07 (Medium):** The system **shall** support **Bid Acceptance**, which automatically triggers the creation of a legally binding digital contract.
- **REQ-08 (Low):** Projects **must** transition through states: Draft -> Open -> In Progress -> Review -> Completed.

2.3 Requirements Engineering & AI Extraction

- **REQ-09 (High):** The platform **shall** allow clients to upload a PDF/DOCX SRS document to initiate a project.
- **REQ-10 (High):** The **Gemini-1.5-Pro API** **shall** be used to extract discrete requirements from the uploaded SRS with 90%+ accuracy.
- **REQ-11 (Medium):** Extracted requirements **must** be editable by the client to adjust priority or phrasing before project start.
- **REQ-12 (Low):** The system **shall** support grouping requirements into **Milestones** for phased delivery.

2.4 GitHub Traceability & NLP Analysis

- **REQ-13 (Critical):** The system **shall** ingest GitHub Webhooks for all

push and

pull_request events to analyze code changes in real-time.

- **REQ-14 (High):** The **Sentence-BERT (SBERT)** model **shall** calculate a semantic similarity score between every commit message and the project requirements.
- **REQ-15 (High):** If a commit score exceeds **0.8**, the requirement status **shall** automatically transition from Not Started to Fulfilled.
- **REQ-16 (Medium):** The system **must** provide a **Manual Sync** button to allow users to retroactively analyze historical commits against new requirements.
- **REQ-17 (High):** Trace links **shall** be persistent in the database, linking a specific Commit SHA to a specific Requirement ID.

2.5 Payment & Escrow Management

- **REQ-18 (High):** The platform **shall** integrate **Stripe Connect** to manage multi-party payments across different countries.

- **REQ-19 (High):** An **Escrow System must** hold client funds securely until the AI Traceability Engine verifies the associated requirement.
 - **REQ-20 (Medium):** Freelancers **shall** be able to request 'Early Milestone Payment' if their Fulfillment Score exceeds 95%.
 - **REQ-21 (Low):** The system **must** generate a downloadable **Invoicing PDF** for every completed milestone payment.
-

3. Non-Functional Requirements

3.1 Performance & Scalability

- **REQ-22 (High):** The API response time for the **Project Dashboard shall** be less than **300ms** under normal load.
- **REQ-23 (Medium):** The NLP Traceability Engine **must** process an ingestion of 50 commits in under **10 seconds**.
- **REQ-24 (Low):** The frontend **shall** support lazy-loading of commit lists to maintain 60FPS scrolling performance.

3.2 Security & Compliance

- **REQ-25 (High):** All API communications **must** be encrypted via **TLS 1.3**.
- **REQ-26 (High):** The system **shall** implement **SQL Injection Protection** and **XSS Sanitization** using SQLAlchemy and DOMPurify.
- **REQ-27 (Medium):** User files (SRS, designs) **shall** be stored in private **AWS S3 Buckets** with Time-Limited Pre-Signed URLs.

3.3 Analytics & Reporting

- **REQ-28 (High):** The system **shall** calculate a real-time **Trust Score** for freelancers based on their requirement fulfillment speed.
 - **REQ-29 (Medium):** A **Drift Detector must** alert clients if the commit history indicates work is deviating from the original requirement scope.
 - **REQ-30 (Low):** The system **shall** provide a **Compliance Audit Report** summarizing all trace links for project hand-off.
-

4. User Interface Requirements

- **REQ-31 (High):** The UI **shall** feature a **Traceability Matrix** visualization showing requirements on one axis and commits on the other.
- **REQ-32 (Medium):** A **Progress Radar Chart** **shall** visualize fulfillment coverage across different requirement categories (Functional vs non-functional).
- **REQ-33 (Low):** The terminal output from the sync process **shall** be accessible to developers for debugging NLP false positives.